Cloud Computing Notes

Today computing services are readily available on demand, just as other utility services.

Users just:

- Pay providers only when they access the services
- No investments in building and mantaining complex infrastructures
- Access services based on their requirements without regard to where the services are hosted.

This model has been referred first as *utility computing* and than as *cloud computing.*

Definition of Cloud Computing (NIST SP800-145)

NIST SP800-145 provides the formal definition of cloud computing as established by the National Institute of Standards and Technology (NIST). The document defines cloud computing as:

> **Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.**

Essential Characteristics

NIST identifies five essential characteristics of cloud computing:

1. **On-demand self-service:** Consumers can unilaterally provision computing capabilities as needed, automatically without requiring human interaction with each service provider.

2. **Broad network access:** Refer to the possibility to access cloud resources from the internet using different devices
3. **Resource pooling:** The provider's computing resources are pooled to serve multiple consumers using a **multi-tenant model**, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a sense of location independence, in that the customer generally has no control or knowledge over the exact location of the provided resources.
    a. **Multi-Tenancy**: multiple users (tenants) access the same application logic simultaneously, each tenant has its own view of the application and are unaware of the others. Multitenant applications isolate data and configuration information
4. **Rapid elasticity:** Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. **Elasticity** is the degree to which a system is able to adapt to workload changes by provisioning and deprovisioning resources in an automatic manner, such that at each point in time the available resources match the current demand as closely as possible. Elasticity is related to Scalability → Horizontal and Vertical scaling.
5. **Measured service:** Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Cloud provider charge customers for the time they utilize the resource or by the amount of MB stored or transferred. Consumers need to monitor the usage of their resources to automate management tasks such as scaling, controlling costs, guarantee application specific SLA and increase resilency. **Resiliency** is the ability to provide and mantain an acceptable level of service in the face of faults and challenges to normal operation (AWS Elastic Load Balancer, AWS Autoscaling, Docker container orchestration) .

Service Models

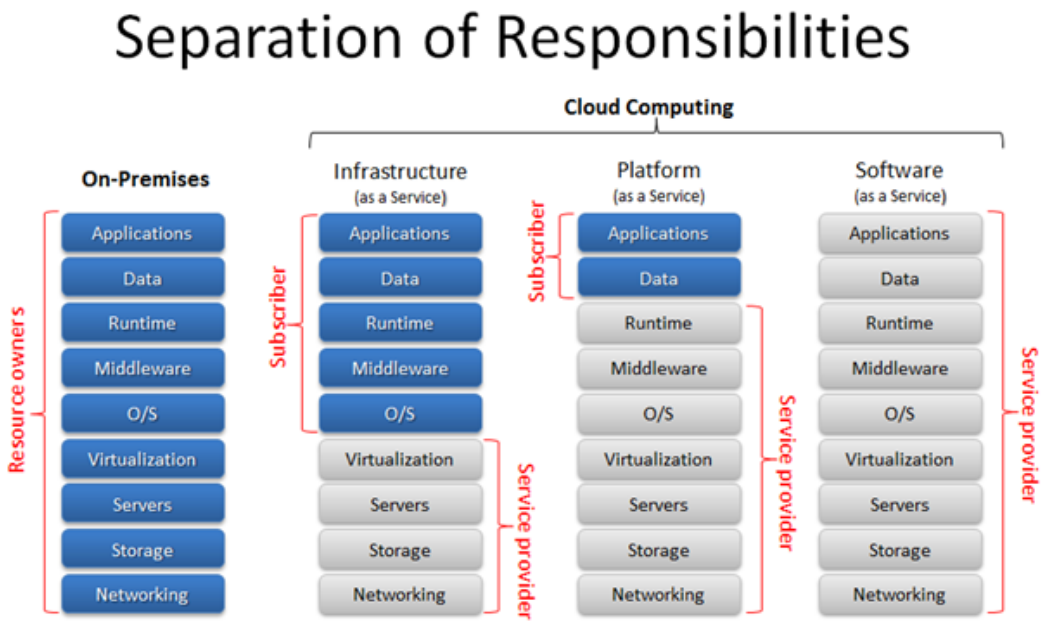
The document also defines three primary service models:

1. **Software as a Service (SaaS):** The capability provided to the consumer is to use the provider's applications running on a cloud infrastructure. The

applications are accessible from various client devices through a thin client interface such as a web browser (E.g. Facebook, Google docs ecc..).

2. **Platform as a Service (PaaS):** The capability provided to the consumer is to deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages, libraries, services, and tools supported by the provider. In few words runtime environment for applications and development and data processing platforms (E.g. Microsoft Azure, Hadoop, Google App Engine ecc...)
3. **Infrastructure as a Service (IaaS):** The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer can deploy and run arbitrary software, which can include operating systems and applications. (E.g. Amazon EC2, S3 ecc..)

Service models also limit the control of the cloud consumer and separate the responsibilities of the cloud consumer and the cloud provider.



Security *of* the cloud - Cloud <u>provider</u> responsibilities

The responsibilities of the provider are:

- Physical security of data centers
- Hardware and software infrastructures
- Network infrastructure (Intrusion detection)

- Virtualization infrastructure (Instance isolation)

Security *in* the cloud - Customer responsibilities

The customer responsibilities are:

- Virtual machines instance operating system
  - Including patching (keep the OS up to date avoiding hacker attacks), maintenance (resource management such as memory, RAM ecc, regular backups ecc)
- OS or host-based firewalls
  - Including intrusion detection or prevention systems
- Applications
  - Passwords, role-based access, etc.
- Account management
  - Login and permission settings for each user
- Network configurations

Deployment Models

Finally, NIST SP800-145 outlines four deployment models:

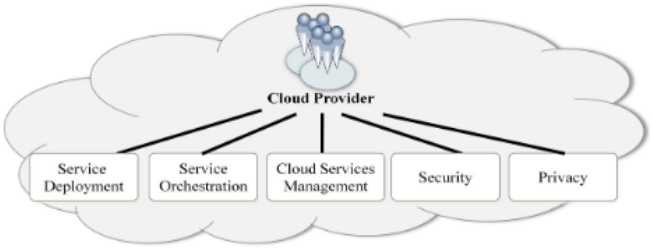
1. **Private Cloud:** The cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple consumers (e.g., business units). It may be owned, managed, and operated by the organization, a third party, or some combination of them, and it may exist on or off premises.
2. **Community Cloud:** The cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns (e.g., mission, security requirements, policy, and compliance considerations). It may be owned, managed, and operated by one or more of the organizations in the community, a third party, or some combination of them, and it may exist on or off premises.
3. **Public Cloud:** The cloud infrastructure is provisioned for open use by the general public. It may be owned, managed, and operated by a business, academic, or government organization, or some combination of them. It exists on the premises of the cloud provider.

4. **Hybrid Cloud:** The cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing between clouds).

Cloud Roles

The term "cloud roles" refer to the distinct responsibilities and functions that different participants in the cloud ecosystem assume. These roles help clarify who is responsible for what, and they guide the design, deployment, and management of cloud services. The primary cloud roles include:

1. Cloud Service Provider (CSP):



- **Role:** The Cloud Service Provider is the entity that offers cloud services to consumers. CSPs own and manage the infrastructure, platforms, or software that they deliver as services over the internet.
- **Responsibilities:**
  - Provisioning, managing, and maintaining the underlying cloud infrastructure (e.g., data centers, servers).
  - Ensuring service availability, security, and performance.
  - Offering cloud service models like IaaS, PaaS, or SaaS to customers.
  - Providing tools for monitoring and managing resources and usage.
  - **Service orchestration**: refers to the composition of system components to support the Cloud Providers activities in arrangement, coordination and management of computing resources in order to provide cloud services to Cloud Consumers

- **Cloud Service Management**: includes all of the service-related functions that are necessary for the management and operation of those services required by or proposed to cloud consumers.

2. Cloud Consumer:

- **Role:** The Cloud Consumer is the individual or organization that uses cloud services provided by the CSP.
- **Responsibilities:**
  - Selecting and purchasing appropriate cloud services based on business needs.
  - Managing and configuring the services they consume (e.g., deploying applications, storing data).
  - Ensuring the security and compliance of their data and applications within the cloud environment.
  - Monitoring resource usage and controlling costs.

3. Cloud Broker:

- **Role:** A Cloud Broker acts as an intermediary between cloud consumers and cloud providers. They help consumers select the best services and might also aggregate, integrate, or customize services.
- **Responsibilities:**
  - **Service Intermediation:** Enhancing existing services by adding value (e.g., managing security, improving performance).
  - **Service Aggregation:** Combining multiple services into one offering to meet consumer needs.
  - **Service Arbitrage:** Offering multiple similar services from different providers, allowing consumers to choose the best option.

4. Cloud Auditor:

- **Role:** A Cloud Auditor is an independent entity that assesses the security, performance, and compliance of cloud services.
- **Responsibilities:**

- Conducting audits to ensure cloud services meet required standards and regulatory compliance.
- Evaluating the effectiveness of the cloud provider's controls and risk management processes.
- Providing audit reports to cloud consumers, regulators, and other stakeholders.

5. Cloud Carrier:

- **Role:** A Cloud Carrier is the intermediary that provides connectivity and transport of cloud services between cloud providers and consumers.
- **Responsibilities:**
  - Ensuring reliable and secure network connectivity.
  - Facilitating data transfer between the consumer's environment and the cloud provider's infrastructure.
  - Managing network-related concerns, such as latency, bandwidth, and availability.

Enabling Technologies

Distributed Systems

> A distributed system is a collection of independent computers that appears to its users as a **single coherent system.**

- Focus on unified usage and aggregation of distributed resources

> A distributed system is one in which components **located at networked computers** communicate and coordinate their actions only by **passing messages**

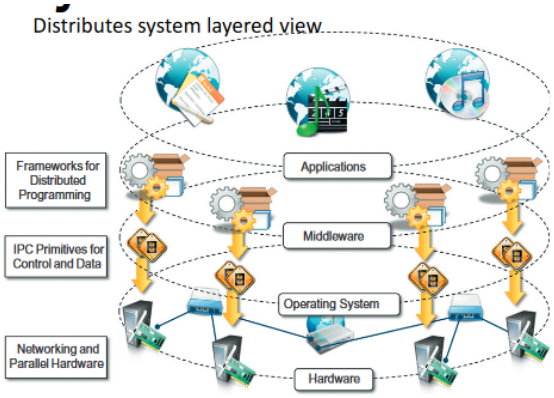
- Focus on the networked nature and communication among processes by passing messages.

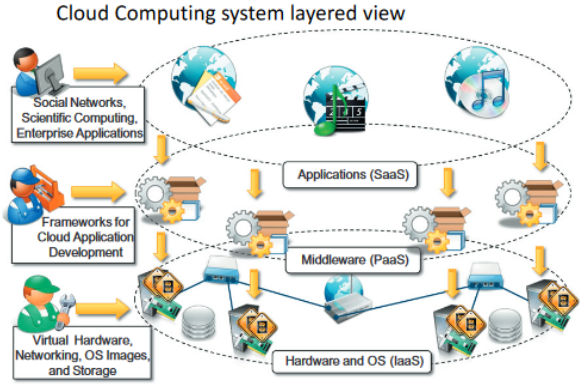
Distributed Computing

It is a computational model: defines how computation is carried on.

- The computation is broken down into units executed concurrently on different computing elements (Processors on different nodes, processors on the same computer, or cores within the same processor)
- Distributed often implies
  - Different locations of the computing elements
  - Heterogeneity in terms of hardware and software features

> 💡 **Cloud Computing is a specialized form of distributed computing that introduces utilization models for remotely provisioning scalable and measured resources.**





Architectural Styles

Mainly used to determine the vocabulary of components and connectors that are used as instances of the style toghether with a set of constraints on how they can be combined.

Define the different roles of components in the distributed system and how they are distributed across multiple machines and what connectors are used.

Software architectural styles

Category	Most Common Architectural Styles
Data-centered	Repository Blackboard
Data flow	Pipe and Filter Batch sequential
Virtual machine	Rule-based system Interpreter
Call and return	Main program and subroutine call/top-down systems Object-oriented systems Layered systems
Independent components	Communicating processes Event systems

Independent Components

Components have their own life cycles and interact each other to perform activities.

Communicating processes:

- Independent process that use Inter Process Comunication (IPC) for coordination management.
- Each process provides services and can use services exposed by the other processes.

Independent Components: Event based systems

- Loosely coupled and connected components
- Each component publish a collection of events other components can subscribe
- Subscriber component provides a callback that will be executed when the event is activated
- Foster the development of open systems: a component can be easily integrated as long as they have compliant interfaces for registering to events

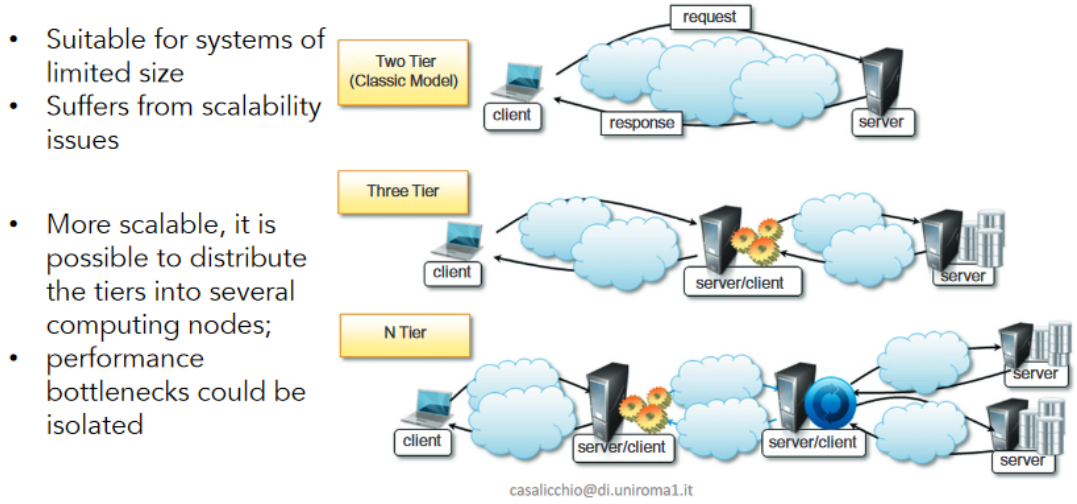
System Architectural Styles

- **Client/Server architecture**

Suitable for many-to-one scenarios. There are three main components: **presentation, application logic and data storage.** We can have two client model:

- Thin-client model:
  - Server → Application logic + Data storage.
  - Client → Presentation
- Fat-client model:
  - Server → Data storage
  - Client → Presentation + Application logic

Presentation, application logic and data maintenance can be seen as conceptual layers which are more appropriately called tiers.



- **Peer-to-peer architecture**

  It is a symmetric architecture → all the components play the same role and incorporate both client and server capabilities.

  Suitable for highly decentralized architecture → Can scale better along the dimension of the number of peers

Inter-process communication (IPC)

IPC is used to either exchange data and information or coordinate the activity of processes.

> IPC is what ties together the different components of a distributed system, thus making them act as a single system.

Model of IPC (the most relevant):

- Message passing
- **Remote procedures call (RPC)**
- **Distributed Objects**
- **Web Services**

Models of communication

- Point-to-point
- Publish-subscribe
- Request-reply

Technologies for Distributed Computing

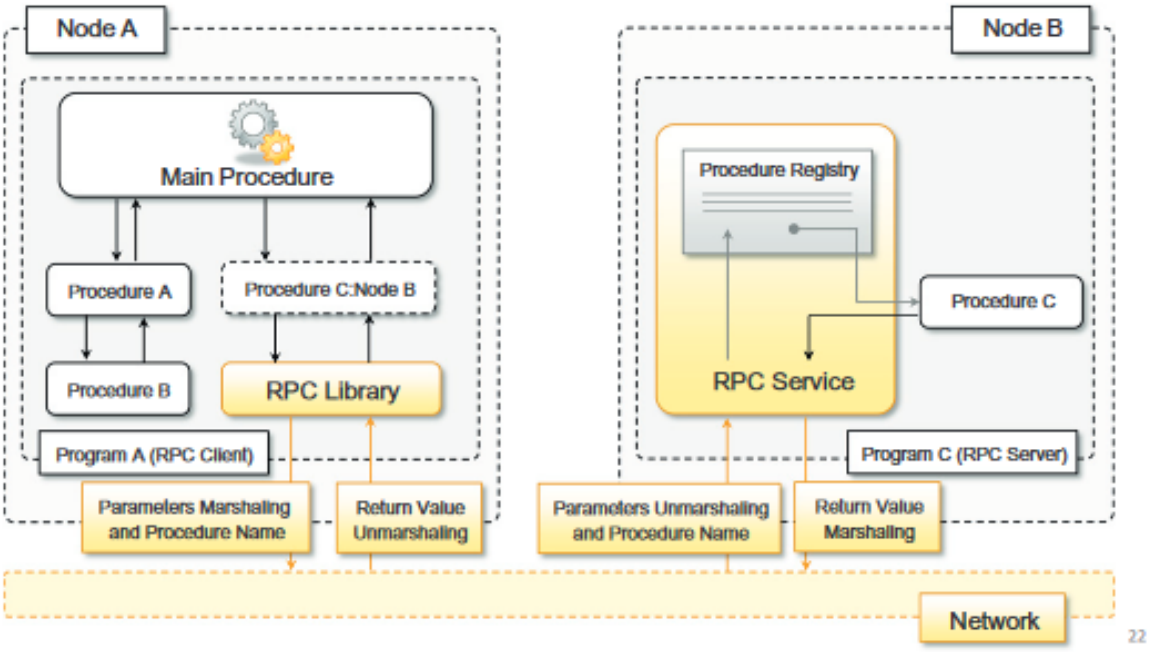
Related to communication:

- **Remote Procedure Call (RPC)**
- **Distributed Objects**
- **Service Oriented Architectures (SOA)**

Related to virtualization:

- Process-level virtualization
- System-level virtualization

Remote Procedure Call



The RPC infrastructure is responsible for marshaling/unmarshaling and for handle the request-reply interaction between C/S.

The implementation is done using RPC API.

RPC is stateless.

Distributed Objects

> Implemementation of the RPC model for the object-oriented paradigm → remote invocation of methods exposed by objects

Each process registers a set of interfaces that are accessible remotely

Client processes can request a pointer to these interfaces and invoke the methods available through them → The object implementing the method became the **server**.

The underlying runtime infrastructure is in charge of transforming the local method invocation into a request to a remote process and collecting the result of the execution.

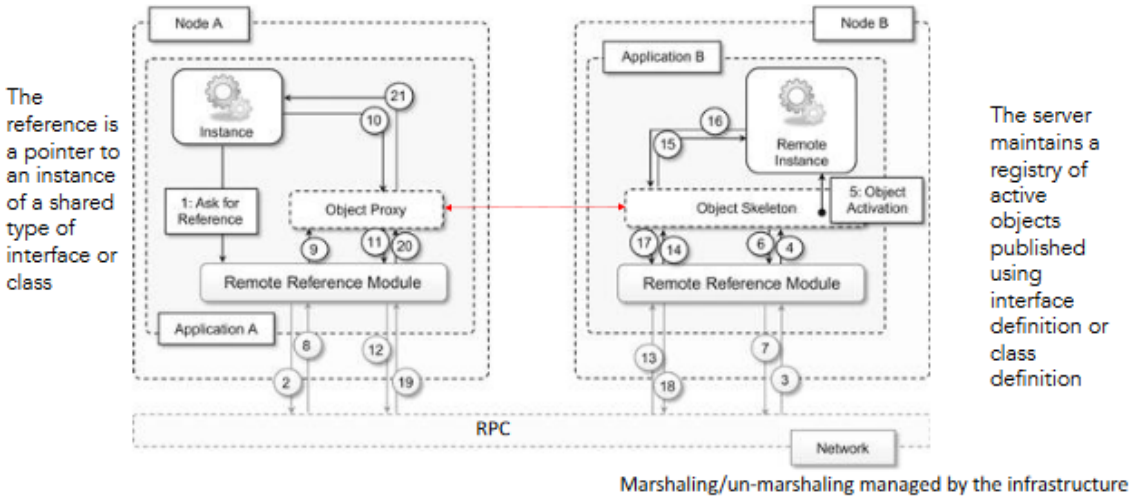
The communication between the caller and the remote process is made through messages.

Distributed object models introduce the complexity of:

- Object state management (by contrast RPC is stateless)
- Lifetime

The methods remotely executed is associated to an object instance:

- Created for the sole execution of the method
- Exist for a limited interval of time, or
- Independent from the existence of requests



If the method invocation foreseen an object to be passed as parameter:

- Marshaling by value
  - A copy of the object is created on the server
  - Inconsistency problems
- Marshaling by reference
  - No duplication of the object – a proxy is created on the server
  - More complex and resource demanding for the infrastructure – remote references have to be tracked

The object activation can be server-based or client-based.

- If **server based** the object has its own life and the remote method invocation is occasional, creation is user controlled.
- If **client based**, the object is created with the purpose of executing the method invoked. The creation is implicit, the lifetime is controlled by the runtime infrastructure.

Service Orientation

Service Oriented Architecture (SOA)

An application is an aggregations of services that are coordinated within a SOA

A service encapsulates a software component and has four main characteristics:

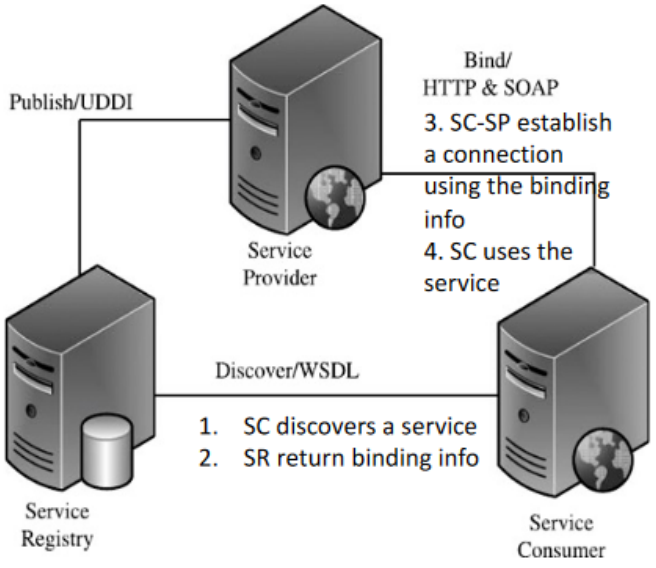
- Boundaries are explicit, interfaces are kept minimal to foster reuse and simplify interaction
- Services are autonomous, can be integrated in different systems at the same time, can handle failures
- Services share schema and contracts, a contract describe the structure of messages the service can send and/or receive and additional constraint on their ordering
- Service (semantic) compatibility is determined based on policies
    - Policies define the capabilities and requirements in term of expressions that must hold true

There are three major roles within SOA:

- **Service provider** → Maintainer of the service, publish the service in a registry together with a service schema and contract
- **Service consumer** → Locate the service metadata in registry, develop the required client components to bind and use the service
- **Service registry** → Store the service descriptors

Service providers and consumers can belong to different organization bodies or business domains.

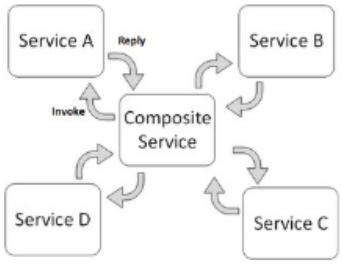
Components can play the roles of both the service provider and service consumer → Aggregating service in workwflows and using them to satisfy consumer research



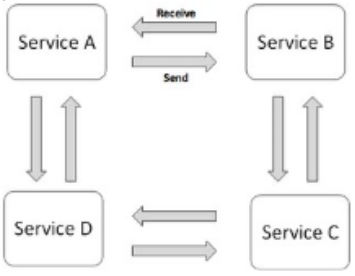
How services can be coordinated and managed?

- **Service Orchestration**: a service orchestrator coordinate and manage the composition and interaction of services in a centralized way
- **Service Choreography**: coordinated interaction (message exchange) of services without a single point of control

Service orchestration represents a single centralized executable business process (the orchestrator) that coordinates the interaction among different services. The orchestrator is responsible for invoking and combining the services.



Service Choreography is a global description of the participating services, which is defined by exchange of messages, rules of interaction and agreements between two or more endpoints. Choreography employs a decentralized approach for service composition.



Web Services - a SOA implementation

Allow the interaction of components that are developed with different technlogies.

A Web Service is exposed as a remote object hosted on a Web server.

Method invocations are transformed in HTTP requests using specific web services:

- **Simple Object Access Protocol (SOAP)**
- **Representational State Transfer (REST)**

Representational State Transfer (REST)

REST is a lightweight alternative to SOAP. The HTTP methods (GET, PUT, POST, DELETE) are used to implement the operation requested by Web Services.

REST Web service follows four basic design principles:

- Use HTTP methods explicitly.
- Be stateless.
- Expose directory structure-like URIs.
- Transfer XML, JavaScript Object Notation (JSON), or both

Basic REST design principle establishes a one-to-one mapping between create, read, update, and delete (CRUD) operations and HTTP methods. According to this mapping:

- To create a resource on the server, use POST.
- To retrieve a resource, use GET.
- To change the state of a resource or to update it, use PUT.
- To remove or delete a resource, use DELETE.

Microservices

Is an architectural style that structures an application as a collection of services that are:

- Highly maintainable and testable
- Loosely coupled
- Independently deployable
- Organized around business capabilities
- Owned by a small team

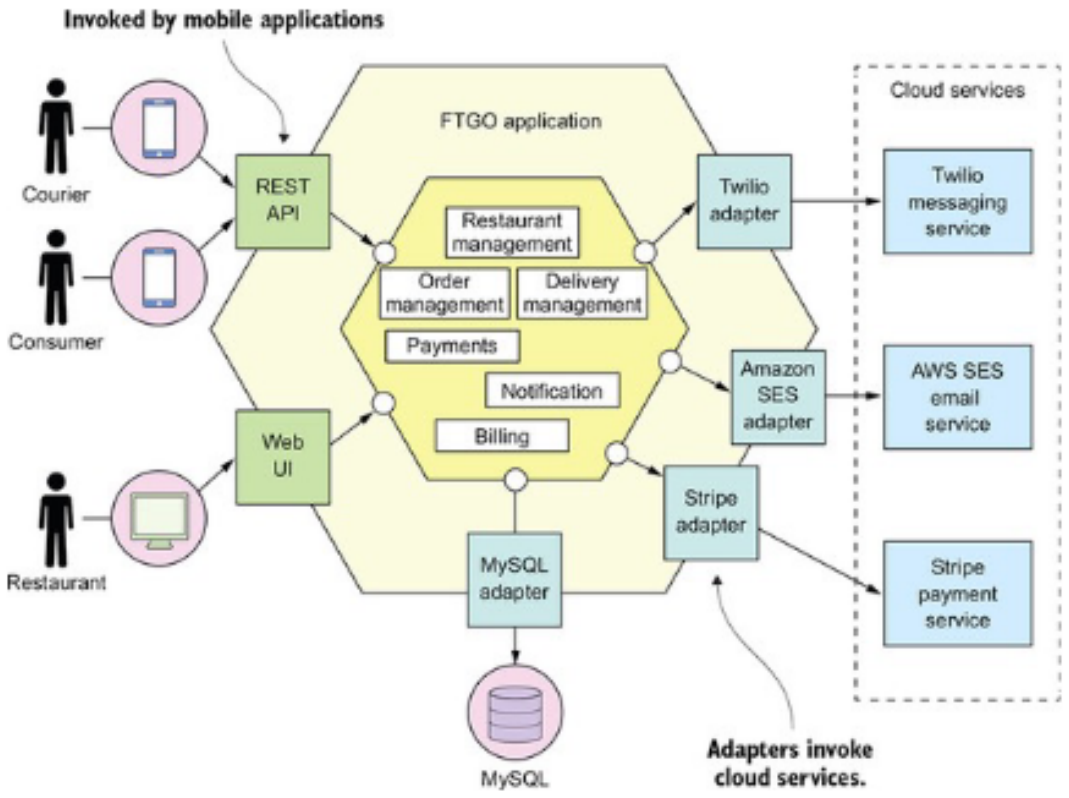
The microservice architecture enables:

- The rapid, frequent and reliable delivery of large, complex applications

- An organization to evolve its technology stack

The Monolithic Architecture Problem:

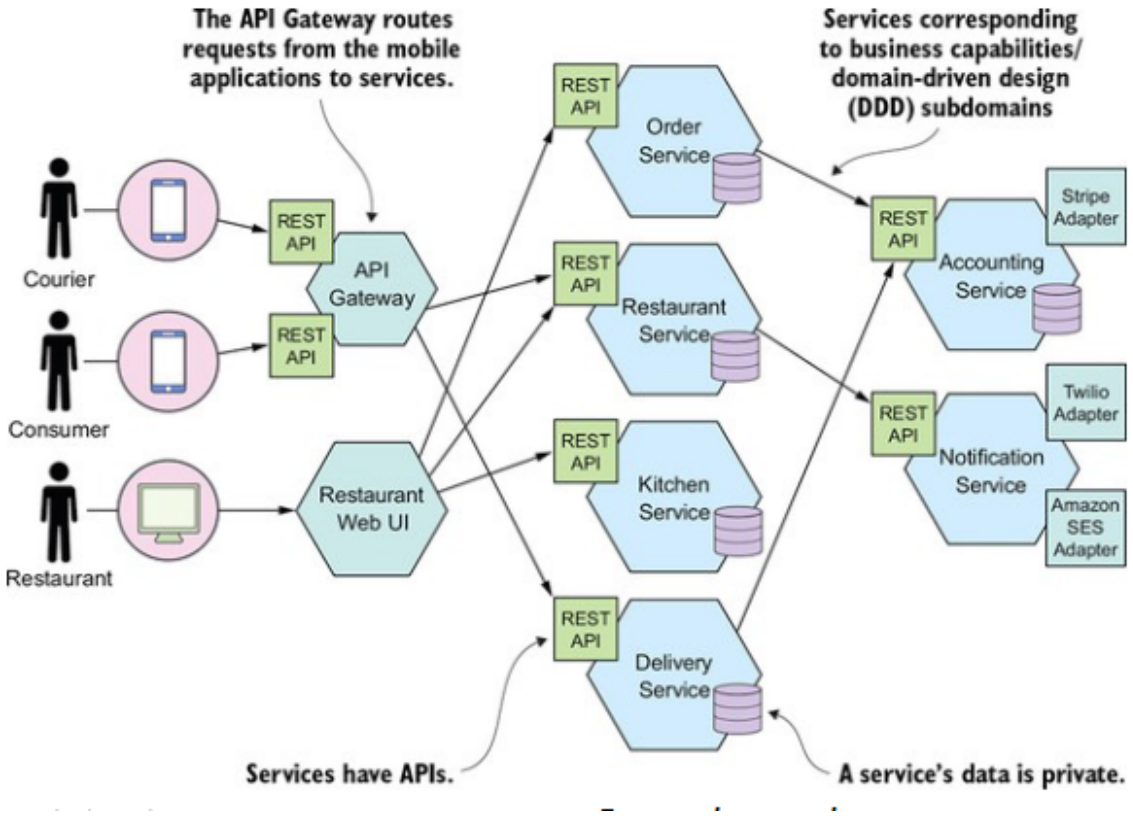
- **Monolithic Design:** Traditional applications are typically built as monoliths, where all the functionality resides in a single codebase and is deployed as a single unit. This design works well initially but leads to several challenges as the application grows.
  - **Pros:**
    - Simple to develop & easy to make radical changes
    - Straightforward to test & deploy
    - Easy to scale (only horizontal scaling)
  - **Challenges:**
    - **Complexity:** As the codebase expands, it becomes harder to manage and understand.

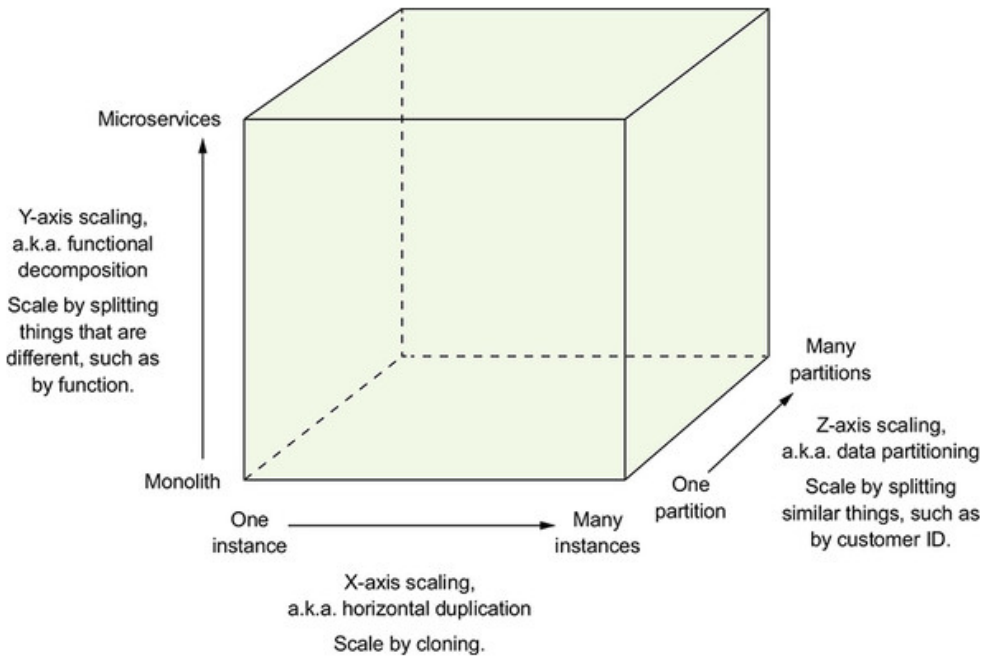


Introduction to Microservices:

- **Definition:** Microservices architecture involves breaking down an application into a set of smaller, loosely coupled, independently deployable services. Each service typically represents a single business capability.
  - **Advantages:**
    - **Scalability:** Each service can be scaled independently.
    - **Independent Deployment:** Teams can deploy services independently without affecting the whole system.
    - **Fault Isolation:** A failure in one service does not necessarily bring down the entire application.
    - **Technology Diversity:** Different services can use different technologies best suited to their specific needs.
    - **Organizational Alignment:** Microservices can align with small, autonomous teams, allowing for faster development cycles and better ownership.

1. **Challenges of Microservices:**
   - **Complexity:** While microservices solve certain problems, they introduce complexity in terms of service discovery (there isn't a well defined algorithm for decomposing a system into services), communication, data consistency, and managing distributed systems.
   - **Operational Overhead:** Monitoring, logging, and deploying microservices are more complex than in a monolithic system.
   - **Data Management:** Ensuring data consistency and managing distributed transactions can be challenging.





Benefits of the Microservice Architecture

- It enables the continuous delivery and deployment of large, complex applications

- Services are small and easily maintained
- Services are independently deployable
- Services are independently scalable
- The microservice architecture enables teams to be autonomous
- It allows easy experimenting and adoption of new technologies
- It has better fault isolation

Drawbacks of the Microservice Architecture

- Finding the right set of services is challenging:
  - There isn't a concrete, well-defined algorithm for decomposing a system into services
  - Planning the scaling of a single service is challenging
- Distributed systems are complex and so development, testing and deployment is challenging
- Deploying features that span multiple services requires careful coordination (among development teams)
- Deciding when to adopt the microservice architecture is difficult

Microservices vs. Service Oriented Architectures

High level similarities → SOA and Microservices structure a system as a set of components

The differences are the following:

▼ Communication mechanism

SOA uses heavyweight technologies (SOAP and other WS* standards)	Microservices uses lightweight, open-source technologies:
SOA uses Enterprise Service Bus, a communication middleware that contain business and messaging processing logic, to integrate the services.	• Message broker
• Slow communication	• REST, gRPC (a modern open source high performance RPC framework)

- Single point of failure
- High configuration and maintenance complexity

▼ Data Handling

- **SOA** applications typically have a global data model and share databases
- In the **Microservice** architecture each service has its own database and data model

▼ Size of the services

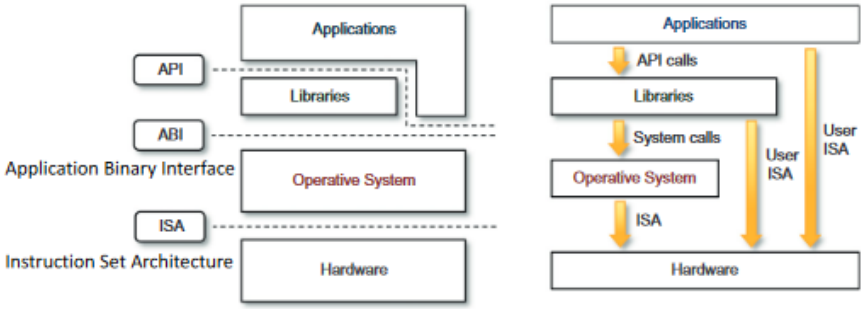
- **SOA** is typically used to integrate large, complex monolithic applications → Application usually consists of a few large services
- **Microservice** architecture use much smaller or ideally tiny services → Application typically consists of 10s or 100s of small services.

Architectural Differences of Microservices vs SOA

Caratteristica	SOA	Microservices
Dimensione dei servizi	Servizi grandi e coesi	Servizi piccoli e focalizzati
Comunicazione	Mediante ESB, accoppiamento moderato	API RESTful, messaging, accoppiamento debole
Gestione dei dati	Database condiviso tra i servizi	Database separati per ogni microservizio
Coordinamento	Orchestrazione centralizzata	Coreografia distribuita
Scalabilità	Meno flessibile, dipende dall'ESB	Elevata, grazie all'indipendenza dei servizi
Adatto per	Integrazione di sistemi legacy e processi complessi	Applicazioni moderne, cloud-native

Virtualization

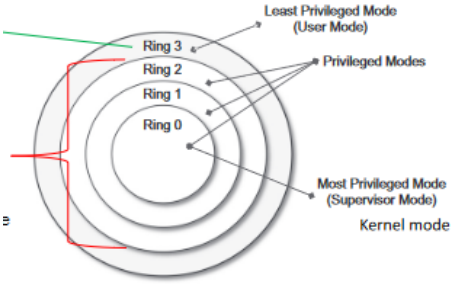
To understand how the different virtualization techniques work it is fundamental to recall the layered structure of a computer system.



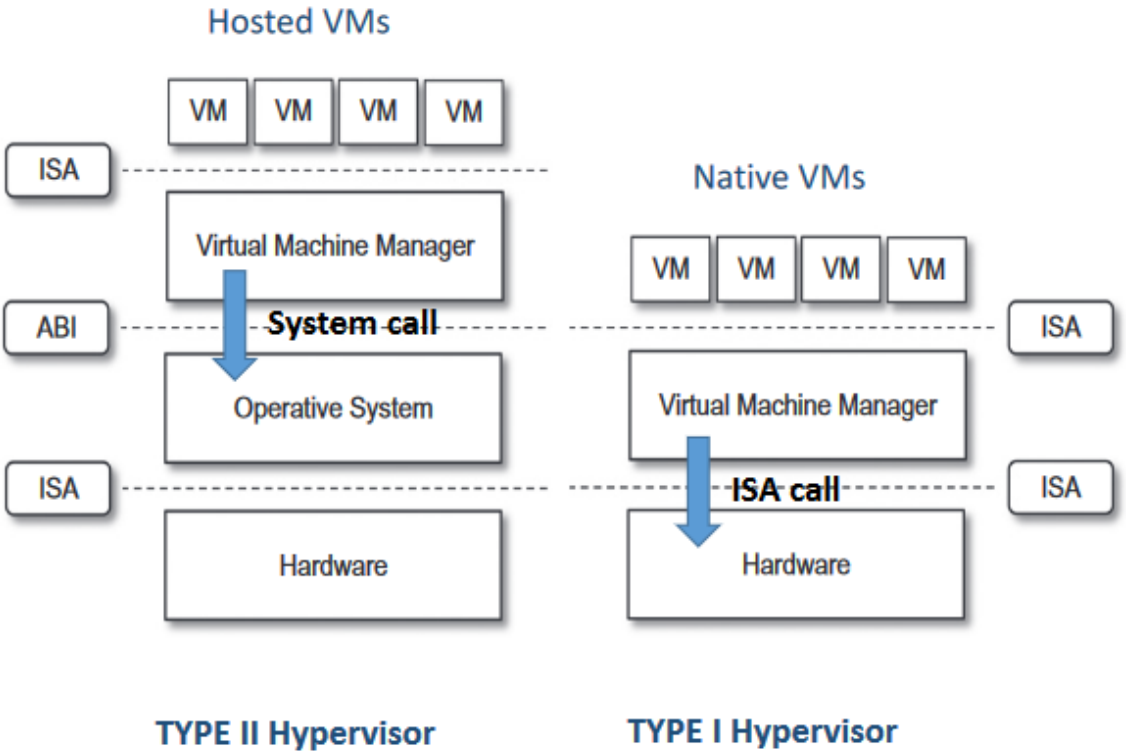
Privileged and Nonprivileged instructions

Nonprivileged instructions can be used without interfering with other tasks.

Privileged instructions are executed under specific restrictions and are mostly used for sensitive operations.



The hyper-visor run on top of the supervisor mode, hence *hyper* (in practice hypervisor run in supervisor mode)



Type I →Data Center

Type II → OS already installed on the device

Virtual Machine Manager

Three properties must be satisfied:

- **Equivalence:** A guest running under the control of a virtual machine manager should exhibit the same behavior as when it is executed directly on the physical host.
- **Resource control**: The Virtual Machine Manager should be in complete control of virtualized resources.
- **Efficiency:** A statistically dominant fraction of the machine instructions should be executed without intervention from the Virtual Machine Manager.

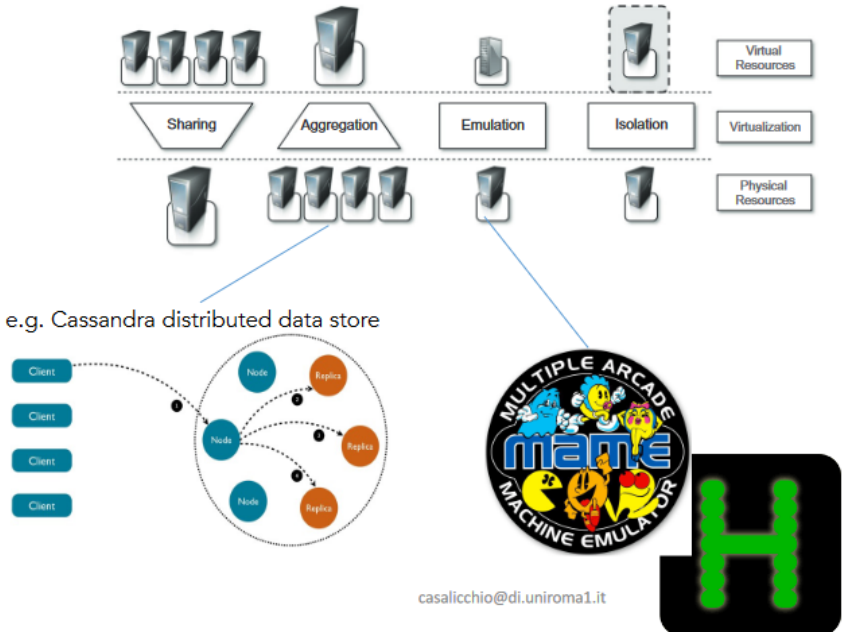
Characteristics of Virtualization Environments

1. Increased security

- The Virtual Machine Manager control and filter the activity of the guest, thus preventing some harmful operations from being performed.
- Resources exposed by the host can then be hidden or simply protected from the guest.
- Sensitive information that is contained in the host can be naturally hidden without the need to install complex security policies.

2. Managed Execution

- **Sharing**: Sharing hardware resources among multiple virtual machines.
- **Aggregation**: Combining multiple hardware resources to form a more powerful virtual resource.
- **Isolation**: Completely separating virtual machines to ensure security and stability.
- **Emulation**: Simulating hardware to run operating systems on different platforms.



3. Portability

> Portability is the possibility to transfer and to use data and applications from a computing platform to another.

A java or python code can be executed on any environment running a JVM or Python VM.

A VM is booted from a VM image that is stored in a specific disk format

The disk format of a virtual machine image is the format of the underlying disk image, e.g.
You can move around VM keeping or converting their format.

A Docker container can run on any platform running the Docker engine.

Full Virtualization

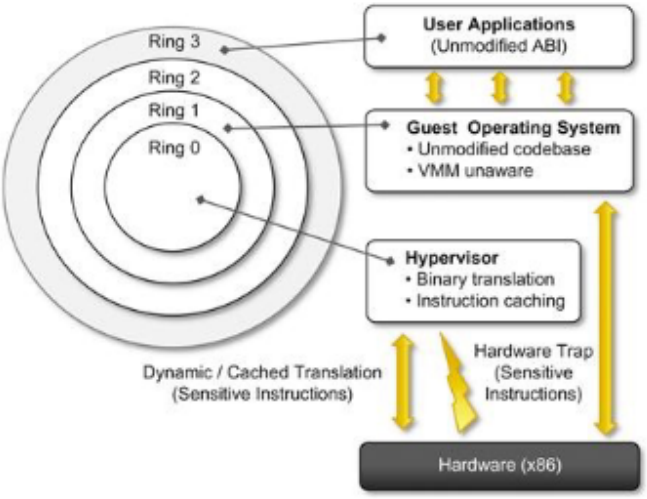
The Virtual Machine Manager (VMM) scans the instruction stream.

- Noncritical instructions run on the hardware directly
- Privileged, control and behavior sensitive instructions are identified and trapped into the VMM, which emulates the behavior of these instructions

The Guest OS is completely decoupled from the underlying hardware → The guest OS is unaware that it is being virtualized

Binary translation is very costly especially for I/O operations.

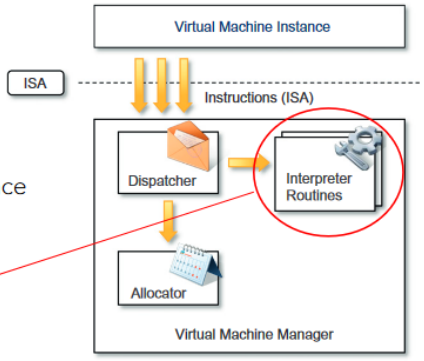
Binary translation is a specific technique in which the VMM translates the binary instruction code of a VM into instructions that the underlying hardware can understand. This process allows for the emulation of the behavior of critical instructions without requiring modifications to the guest operating system.



Hardware Assisted Virtualization

In hardware-assisted virtualization, both privileged and non-privileged instructions can run directly on the hardware, reducing the need for trapping and translating instructions.

- Examples of hardware-assisted virtualization are
  - the extensions to the x86-64 bit architecture introduced with Intel VT
  - AMD V
- Reduce the performance penalties experienced by emulating x86 hardware with VMM
  - by design the x86 architecture did not meet the Equivalence requirements
    - 17 sensitive instructions called in user mode
    - early products were using binary translation to trap some sensitive instructions and provide an emulated version

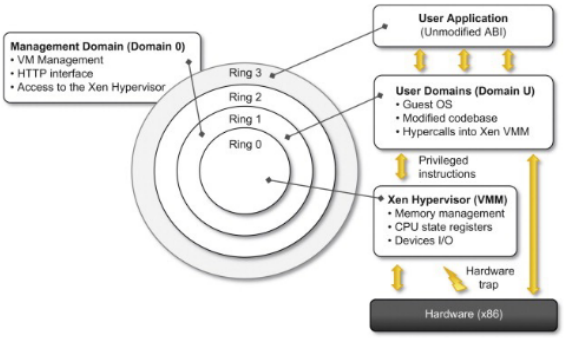


Paravirtualization

- **Definition**: Paravirtualization is a virtualization technique where the guest operating system is modified to be aware of the hypervisor, allowing for more efficient communication between the guest OS and the hypervisor.
- **How It Works**: In paravirtualization, the guest OS makes hypercalls (similar to system calls) to the hypervisor for resource management, which reduces the overhead associated with instruction trapping. This approach usually results in better performance compared to traditional full virtualization but requires changes to the guest OS.

Para-virtualized OS are assisted by an intelligent compiler to replace the nonvirtualizable OS instructions by hypercalls.

- The guest must support paravirtualization
- Require HW assisted virtualization
- Prevent performance losses



Autonomic Computing

> Autonomic computing systems are computing systems that can manage themselves given high-level objectives from administrators.

Autonomic Manager

An **Autonomic Manager (AM)** is a core component of an autonomic computing system. It is responsible for managing a specific resource or set of resources within the system autonomously, according to the self-managing properties described in the vision of autonomic computing (self-configuration, self-healing, self-optimization, and self-protection).

Roles and Functions of an Autonomic Manager:

- **Monitoring**: Continuously observes the status and performance of the managed resources. This involves collecting data on system behavior, such as resource usage, error rates, or response times.
- **Analysis**: Interprets the collected data to identify patterns, trends, or anomalies. This analysis helps the autonomic manager understand the current state of the system and predict potential issues or opportunities for optimization.
- **Planning**: Based on the analysis, the autonomic manager devises a plan to adjust the system. This could involve reconfiguring resources, repairing faults, reallocating tasks, or enhancing security measures.
- **Execution**: The manager carries out the planned actions to adjust the system. This could mean deploying new configurations, restarting services, migrating workloads, or other corrective measures.

The autonomic manager operates based on high-level policies defined by human administrators, ensuring that its actions align with the overall goals and constraints of the system.

MAPE-K Cycle

The **MAPE-K cycle** is a conceptual framework that describes how an autonomic manager operates. It stands for **Monitoring, Analysis, Planning, Execution, and Knowledge**, which are the four phases of the cycle. The "K" stands for "Knowledge," which is the shared repository or data that the autonomic manager uses throughout the cycle.

1. **Monitoring**:
    - The autonomic manager collects data from the system. This data could include performance metrics, logs, error reports, and other relevant information. Monitoring provides the necessary input for the analysis phase.
2. **Analysis**:
    - In this phase, the autonomic manager analyzes the data collected during monitoring. The goal is to understand the current state of the system, identify problems, or spot opportunities for improvement. The analysis might involve statistical methods, pattern recognition, or machine learning techniques.
3. **Planning**:

- Based on the analysis, the manager creates a plan of action. This plan outlines the steps needed to adjust the system to achieve desired outcomes, such as fixing an issue, enhancing performance, or adapting to new requirements.

4. **Execution**:
   - The autonomic manager implements the plan. This involves making changes to the system, such as reconfiguring settings, deploying updates, reallocating resources, or other actions necessary to address the identified issues or optimize performance.
5. **Knowledge (K)**:
   - Throughout the MAPE-K cycle, the autonomic manager relies on a knowledge base, which contains information about the system, its components, historical data, policies, and best practices. This knowledge is crucial for making informed decisions during the analysis and planning phases. The knowledge base can also be updated as the system learns from new data and experiences.

The Four Self- Properties:

Autonomic computing systems are characterized by four key properties:

- **Self-Configuration**: The system should be able to automatically configure itself based on high-level policies or goals. This includes the installation and configuration of software, hardware, and network components.
- **Self-Healing**: The system should detect, diagnose, and repair localized problems automatically. This includes recovering from failures and minimizing downtime.
- **Self-Optimization**: The system should continuously monitor and optimize its performance by reallocating resources as needed to meet performance goals.
- **Self-Protection**: The system should anticipate, detect, and protect itself from attacks or failures. This includes implementing security policies and responding to potential threats.

Autonomic Manager Configuration Policies

In autonomic computing policies are crucial for configuring an autonomic manager to express high-level objectives. Several types of policies that can be used:

- **Goal Policies**: These specify the desired outcomes without dictating how to achieve them. They are declarative and focus on the end state that the system should reach.
- **Utility Function Policies**: These policies use utility functions to express objectives, where different states or actions are assigned utility values. The system aims to maximize or optimize the utility.
- **Action or Rule-Based Policies**: These policies specify specific actions that should be taken when certain conditions are met. They are more procedural and directive than goal policies.
- **Event-Condition-Action (ECA) Policies**: These specify actions to be performed in response to specific events when certain conditions hold. ECA policies are commonly used in reactive autonomic systems.

Monitoring

Importance of Monitoring:
Monitoring is a critical component of autonomic computing because it provides the data needed for the autonomic manager to make informed decisions. The autonomic manager relies on monitoring to understand the current state of the system and to detect deviations from desired behaviors, which then triggers appropriate self-management actions such as self-configuration, self-optimization, self-healing, or self-protection.

Types of Monitoring Techniques:

- **Periodic Monitoring**: Involves sampling the system state at regular intervals. This method is straightforward but may miss rapid changes between sampling periods.
- **Event-Based Monitoring**: Focuses on detecting specific events or changes in the system state. This approach is more responsive and can be more efficient by reducing unnecessary data collection.
- **Adaptive Monitoring**: Adjusts the frequency and granularity of monitoring based on the system's current state or context. For example, the system may increase monitoring intensity when it detects potential issues.

- **Statistical Monitoring**: Involves the use of statistical techniques to detect anomalies or trends over time. This can include techniques like moving averages, trend analysis, or threshold-based alerts.

Planning

In the planning phase of autonomic computing, a **model-driven approach** involves using abstract models to guide the generation of plans or strategies for system adaptation. These models represent various aspects of the system, such as its architecture, behavior, or environment, and can be used to simulate different scenarios, predict outcomes, or generate action plans.

Key aspects of the model-driven approach include:

- **Use of System Models**: Abstract representations of the system, such as architectural models, performance models, or workload models, are used to predict the effects of potential changes or adaptations.
- **Model Transformation**: Involves transforming these abstract models into executable plans or workflows that can be implemented by the autonomic manager.
- **Simulation and Optimization**: The models can be used to simulate different configurations or strategies to find the optimal solution that meets the high-level objectives defined by the policies.

Degree of Autonomicity - AC Adoption Model Levels

The paper describes different **degrees of autonomicity**, which refer to the levels at which a system can manage itself. These degrees are often conceptualized as a spectrum from manual management to full autonomic operation:

- **Level 0 - Manual Management**: No autonomic capabilities; all decisions and actions are performed manually by human operators.
- **Level 1 - Basic Automation**: Highly skilled staff utilise monitoring tools and then make the require changes manually to control system elements
- **Level 2 - Managed**: The system's monitoring tools collage information in an intelligent enough
  way, the systems administration burden is reduced

- **Level 3 - Predictive**: More intelligent monitoring than Level 2. System behavior patterns are recognized. Suggest actions approved and carried out by IT staff.
- **Level 4 - Adaptive**: The system uses the types of tools available to Level 3 and is more able to take action. Human interaction is minimized and it is expected that the performance is tweaked to meet SLA.
- **Level 5 - Fully Autonomic**: The system is fully self-managing in all aspects, including self-configuration, self-optimization, self-healing, and self-protection, without requiring any human input.

These degrees reflect the progression from a system that is entirely human-managed to one that is capable of fully autonomic operation.

Automation & Orchestration

IT Automation vs Autonomic computing

Autonomic computing → Keep humans out of the loop

Automation → Reduce human interaction and manual processes

IT Operation Automation

How

- Software tools and technologies to automate various organizational and operational IT processes.

What

- Automating repetitive tasks, orchestrating workflows, and integrating automation into existing systems and applications.

Benefits

- Improved efficiency and productivity: reduce the time and effort required to perform manual tasks. This frees up IT staff to focus on more strategic and value-added activities.
- Reduced human error: eliminate the risk of mistakes caused by manual intervention. This can lead to more accurate and consistent results.
- Saved time and costs: minimize the burden of repetitive and time-consuming tasks. Free up IT staff helps reduce the overall operational costs

of the organization.

Ansible (Practical example of Automation)

Ansible is an open source IT configuration management, deployment and orchestration tool.

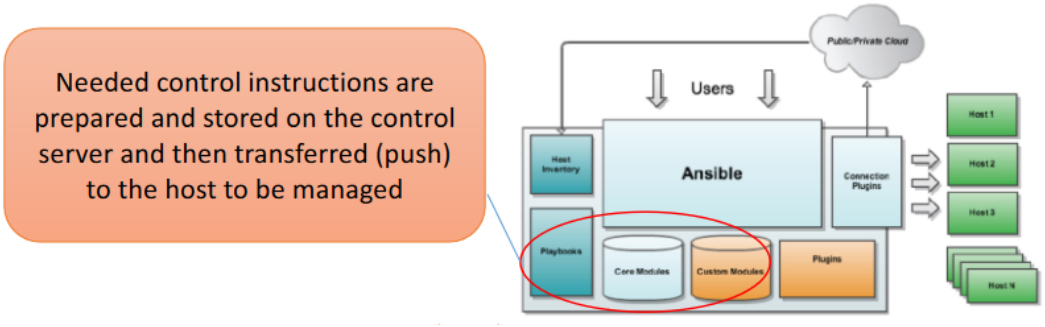
It is an **agentless** tool that runs in a **'push' model** (but can be reverted in pull model).

Agentless

No software is required to be installed on remote machines to make them manageable.

No overhead when management is not running.

Ideal for high-security environments or high-performance cases where there are concerns about stability or permanence of a management agent.



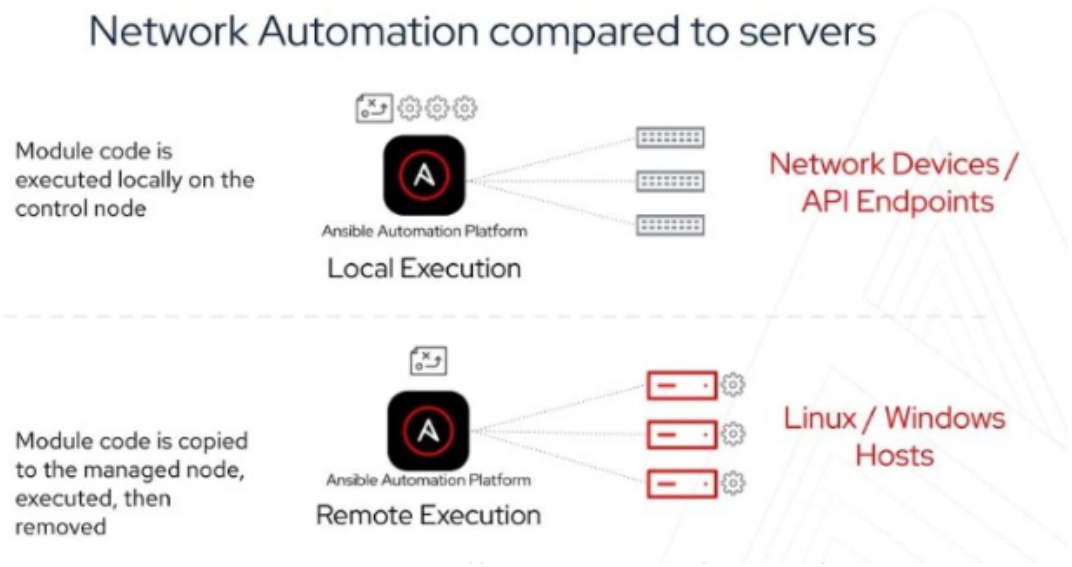
Push Model

Manages remote machines by using remote management framework already existing on the platforms.

Needed code (called **Ansible modules**) are passed to remote machines (**push**).

Remote machines cannot see or affect how other machines are configured.

> **Not all the nodes can run code**



Playbooks

Ansible playbooks allow simple configuration management and multi-machine deployment system, they use a YAML syntax and are repeatable and reusable.

Playbooks can:

- Declare configurations
- Orchestrate steps of any manual ordered process, in multiple sets of machines, in a defined order
- Launch tasks synchronously (default) or asynchronously (for long running task)

Playbooks consist of series of 'plays' that define automation across a set of hosts, known as inventory.

Each play consists of multiple 'tasks', that can target one, many, or all of the hosts in the inventory.

Each task is a call to an Ansible module - a small piece of code for doing a specific taks.

Orchestration

Automation → Setting up one task to run on its own

Orchestration → Automating a series of individual tasks to work together as a process or workflow

Orchestration concept could be specialized depending on the context

- Service orchestration (we already presented this concept)
- IT infrastructure orchestration
- Container orchestration

Container Orchestration

Allows cloud and application providers to define how to select, deploy, monitor, and dynamically control the configuration of multicontainer packaged applications in the cloud.

Is concerned with the management at runtime to support the Deploy, Run, and Maintain phases

Usually offer the following main features: resource limit control, scheduling, load balancing, health check, fault tolerance, and auto-scaling.

Kubernetes (K8s)

Kubernets is a platform for managing containerized workloads and services, but what does it means to manage containerized workload?

- **Service discovery and load balancing**
- **Storage orchestration**; automatically mount storage
- **Automated rollouts and rollbacks**; automatically bring the deployed containers in a desired (described) state
- **Automatic bin packing**; fit containers onto clusters nodes to make best use of resources; for each container is give the needed CPU and memory
- **Self healing**; restart, replace, kill
- **Secret and configuration management**

K8s Objects

K8s objects are persistent entities in K8s and are used to represent the state of the cluster. Specifically, they can describe:

- What containerized applications are running (and on which nodes)
- The resources available to those applications
- The policies around how those applications behave, such as restart policies, upgrades, and fault-tolerance

A K8s object is a "record of intent"-- once the object is created, the K8s system will constantly work to ensure that object exists.

An object contains two fields:

- **spec** → describe the **desired state**
- **status** → describe the **current state** of the object

Pods

Pods are the smallest deployable units of comuting that can be created and managed by K8s.

Models an application-specific 'logical-host': it contains one or more application containers which are relatively tightly coupled, and share storage resources, network resources and a specification on how to run the containers.

A pod's contents are alway co-located and co-scheduled, and run in a shared context

The shared context of a Pod is a set of Linux namespaces, cgroups

A Pod is similiar to a set of containers with shared namespaces and shared filesystem volumes.

K8s Nodes

Nodes are virtual or physical machine, member of a K8s cluster

Each node is managed by the control plane and contains the services necessary to run Pods

Nodes run containerized workload (Pods)

Nodes became member of a K8s cluster in two way:

- The kubelet on a node self-registers to the control plane (API-server)
- A human user manually adds a Node object to the API-server

Once a Node object is created the control plane checks whether the new Node object is valid (all the service are running). If the node object is valid it could run a Pod

K8s node status

The node status is described by:

- Address (Hostname, IP)
- Condition (Ready, DiskPressure, MemoryPressure, PIDPressure, NetworkUnavailable)
- Capacity → indicates the resources available on the node: CPU, memory and the max number of pods that can be scheduled onto the node
- Allocable → indicates the amount of resources on a Node that is available to be consumed by normal Pods
- Info → general information about the node, such as kernel version, Kubernetes version, container runtime details and which operating sustem the node uses

K8s cluster

Formed by **Control plane** (that runs on a dedicated server or VM) and **nodes** (that is a VM or a physical server)

K8s Control Plane components

- **kube-apiserver →** front-end of the K8s control plane
- **etcd** → consistend and highly-available key value store used as K8s'backing store for all cluster data
- **kube-scheduler** → watches for newly created Pods with no assigned node and selects a node for them to run on
- **kube-controller-manager** → Runs controller processes (Node controller, Job controller ecc...)

- **cloud-control-manager** → to link a K8s coìluster with a specific cloud provider's API

Controllers

Controllers are control loops that watch the state of a K8s cluster, then make or request changes where needed. A controller tracks at least one Kubernetes resource type and will send messages to the API server that have useful side effects.

Scheduler

Scheduling refers to making sure that Pods are matched to Nodes so that Kubelet can run them. A scheduler watches for newly created Pods that have no Node assigned.

Kube-scheduler

Is the default scheduler. → For every newly created pod or unscheduled pods selects an optimal node, nodes that meet the scheduling requirements for a Pod are called feasible nodes.

The scheduler finds feasible Nodes for a Pod (**Filtering**), then runs a set of functions to score the feasible Nodes (**Scoring**), picks a Node with the highest score among the feasible ones to run the Pod. Finally notifies the API server about his decision in a process called **binding**.

K8s Node components

- **kubelet** → it is an agent running on each node, it controls that containers described in a PodSpec (input for the kubelet) are running and healthy
- **kube-proxy** → it is a network proxy that runs on each node. It maintains network rules on nodes, these rules allow network communication to cluster's Pods from network sessions inside or outside of the cluster
- **container runtime** → the software that is responsible for running containers (Docker or CRI-O)

Autoscaling

Autoscaling refers to the capability of automatically adjusting the number of computational resources (such as VMs, containers, etc.) allocated to an application, based on the current workload.

- **Types of Scaling**:
  - **Vertical Scaling**: Adjusting the resources (CPU, memory) within a single instance.
  - **Horizontal Scaling**: Adding or removing instances to handle the load, also known as scale-out/in.

Autoscaling Algorithms

1. **Threshold-Based Algorithms**:
   - **Reactive Approach**: Scaling actions are triggered when resource utilization crosses predefined thresholds. For instance, if CPU usage exceeds 70%, a new instance is added.
   - **Proactive Enhancements**: Multiple thresholds can be set to anticipate load changes more effectively, although these are heuristics and may not be optimal, potentially leading to over or under-provisioning.
2. **Model-Based Algorithms**:
   - Uses mathematical models to determine scaling actions based on system behavior, performance metrics, and workload intensity. This method can be optimal and is triggered by specific events related to these metrics.
3. **Proactive Algorithms**:
   - **Prediction-Based**: Predicts future workload and resource utilization (e.g., for the next 5-10 minutes) and takes scaling decisions based on these predictions.

Threshold based algorithms

Triggering Autoscaling actions

$$
\begin{gathered}
u_t = \text{utilization of the system at time } t \\
U \text{ scaling out threshold} \\
L \text{ scaling down threshold} \\
u_t \text{ can be measured or estimated with a model}
\end{gathered}
$$

- **Triggering Actions**: Scaling decisions are based on current or predicted resource utilization against predefined thresholds. Conservative strategies require sustained threshold violation over several time windows before scaling.
  - Predictive → scale if the predicted value u_{t+1} violate the threshold

$$u'_{t+1} = \begin{cases} \beta u'_t + (1-\beta)u_t & \text{if } t > 0 \\ 0 & \text{otherwise} \end{cases}$$

- **Sizing Scaling Steps**:
  - Scaling quantity can be either fixed or adaptive.

  Adaptive strategy

  Define an upper and lower bound for step size s for both scale-out/in, let say L_t, U_t

  Determine s as:

$$\begin{cases} s_t = (1-\alpha)L_t + \alpha U_t, & \text{SCALE-IN} \\ s_t = \alpha L_t + (1-\alpha)U_t & \text{SCALE-OUT} \end{cases}$$

Autoscaling in AWS

1. **Dynamic Scaling Mechanisms**:
   - **Simple and Step Scaling**: Triggered by thresholds on performance metrics, allowing predefined adjustments in the number of instances.
   - **Target Tracking Scaling**: Targets a specific value for a metric (like CPU utilization) and adjusts the number of instances to maintain this target.

```
do {
Listen for the occurrence of an Alarm (event associated
If (alarm occur) then
pass the control to the component that execute the scal
end_if
} while true;
```

   - **Warm-up Period**: Newly launched instances are not included in metrics calculations until they are fully operational, avoiding premature scaling

actions.

2. **Scaling Adjustment Types**:
   - **Change in Capacity**: Directly increases or decreases the number of instances.
   - **Exact Capacity**: Sets a specific number of instances.
   - **Percent Capacity**: Adjusts the capacity based on a percentage change.

Aggregated Metrics

At a given instant of time t the 'Performance Metric value' used by scaling policy to take a scaling decision is the aggregated of the 'Performance Metrice valueì values at time t collected from all the instances for that metric.

For example, if there are 5 instances, the metric 'CPU utilization is:

$$\frac{1}{5}\sum_{i=1}^{5}\text{CPUutil}(t, VM_i)$$

Autoscaling in Kubernetes

1. **Horizontal Pod Autoscaling (HPA)**:
   - Adjusts the number of pods based on CPU utilization. The control loop periodically checks the CPU usage of each pod, and if it deviates from the target utilization, the number of pods is adjusted accordingly.

HPA Controller algorithm → returns the number of pods to be deployed

$Input: U_{\text{target}}, ActivePods$ → Target utilization and the set of active Pods

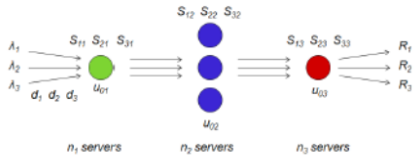
$Output: P$ → Number of Pods to deploy

```
while true do
    for all i in ActivePods do
        U_i = getCPUUtilization(i)
        U = U union U_i
    end for
    P = ceil(sum(U)/U_target)
```

```
    wait(tau) #The control loop period, tau usually is 30s
end while
```

Advanced Autoscaling Techniques

1. **Adaptive, Model-Driven Autoscaling**:
    - Combines predictive models with dynamic adaptation to optimize resource allocation.



$$u_j = u_{0j} + \sum_i \lambda_{ij} S_{ij}, \qquad \forall j$$

$$R_i = d_i + \sum_j \frac{S_{ij}}{1 - u_j}, \qquad \forall i$$

$$\lambda_{ij} = \frac{\lambda_i}{n_j}$$

2. **Online VM Auto-Scaling Algorithms**:
    - **Joint VM Auto-Scaling and VM-to-PM Packing**: Focuses on dynamically adjusting the number of VMs and placing them efficiently on physical machines to optimize resource usage while meeting performance requirements.

Symbol	Definition
i	application type; $i = 1, \ldots, I$
j	VM class type; $j = 1, \ldots, J$
k	resource type; $k = 1, \ldots, K$
m_{ij}	number of class j VMs assigned to serve application i
m^*_{ij}	avg. number of class j VMs assigned to serve application i
$1/\mu_i$	average service time of an application i request
a_{jk}	amount of resource k required by a class j VM
w_{ij}	max no. of application i's requests served by a class j VM
s	feasible configuration vector that satisfies condition (1)
S	set of undominated feasible configuration vectors
A_k	total amount of resource k in a PM
β	total number of PMs in a DC
λ_i	avg. arrival rate of application i's requests
ϕ_s	fraction of PMs being assigned with configuration s
ϕ^*_s	avg. fraction of PMs being assigned with configuration s
ρ	average fraction of the utilized PMs
Q_i	virtual queue size associated with application i
V_j	virtual queue size associated with VM j

$$\min_{\{m^*_{ij}\},\{\phi^*_s\},\rho} \rho,$$

subject to

$$m^*_{ij} \geq 0, \ \forall (i,j), \quad \phi^*_s \geq 0, \ \forall s,$$

$$\lambda_i/\mu_i \leq \sum_j w_{ij} \cdot m^*_{ij}, \quad \forall i,$$

$$\sum_i m^*_{ij}/\beta \leq \sum_{s \in S} s_j \phi^*_s, \quad \forall j,$$

$$\sum_{s \in S} \phi^*_s = \rho.$$

Problem Statement and Solutions

- **Challenges**: Handling varying demand loads, efficient resource allocation, and maintaining system stability.

- **Solution Approaches**:
  - **Shadow Routing Algorithm**: Minimizes the activation of additional servers by solving linear programming problems that optimize server usage while keeping virtual queues stable.

    05-Autoscaling.pdf

  - **App to VM and VM to PM Mapping**: Strategies for effectively mapping applications to VMs and VMs to physical machines.

Testing Scalability

Testing Scalability: Concepts and Approach

- **Scaling In/Out**:
  - **Scaling In**: Reducing the number of active resources when demand decreases.
  - **Scaling Out**: Increasing the number of active resources as demand increases.
- **Continuous Load Increase/Decrease**:
  - Load should be increased continuously without cooldown periods between test runs.
  - Typical testing phases include:
    1. **Warm-Up (WU)**: Initial period to prepare the system.
    2. **Ramp-Up (RU)**: Gradual load increase.
    3. **Steady State (S)**: Sustained maximum load.
    4. **Ramp-Down (RD)**: Gradual load decrease.
- **Shell Script for Load Testing**:
  - A simple shell script example provided for testing with Apache Benchmark (ab) to simulate varying loads.

    - For more control, sophisticated tools like **JMeter** are recommended.
- **Tools for Load Testing**:
    - **JMeter**: Full control over testing, suitable for various protocols.
    - **Artillery and Loadster**: Easier configuration but with limited control and potential risks of overloading cloud services like AWS.

Reproducing User Behavior in Tests

- **Challenges**:
    - Capturing realistic user behavior is critical for accurate testing.
- **Solutions**:
    1. **Representative Requests**: Use a set of requests with assigned probabilities, distributing the load accordingly.
    2. **Selenium WebDriver**: Simulates user interactions for more realistic testing.

Testing Configurations

- **Typical Setup**:
    - A multi-VM (Virtual Machine) environment with load balancers and autoscaling mechanisms.
    - Ensure testing remains within cloud service provider limits (e.g., AWS) to avoid bans.
- **Complex Applications**:
    - For multi-tier applications (front-end, back-end, database), each component's scalability must be tested separately, adhering to the same principles (WU, RU, S, RD).

Metrics for Scalability Testing

- **Client-Side Metrics** (often less useful):
    - Response time, latency, throughput, traffic, and availability.
    - These metrics are influenced by external factors (e.g., internet traffic) and don't necessarily reflect the server or cloud side performance.

- **Cloud-Side Metrics**:
  - **IaaS (Infrastructure as a Service)**: Focus on CPU utilization, disk throughput, network traffic, and the number of active instances.
  - **PaaS (Platform as a Service)**: Metrics like invocation counts, execution duration, error rates, and concurrent executions are critical.
- **Workload Bound Metrics**:
  - Choose metrics based on the workload type:
    - **CPU Bound**: CPU utilization.
    - **Memory Bound**: Memory utilization.
    - **Network Bound**: Number of connections, incoming/outgoing traffic.
    - **Disk Bound**: Read/Write transactions per second.

Setting Threshold Values

- **Heuristic Process**:
  - Threshold values often follow a rule of thumb—typically 66% of the system's maximum capacity.
  - Adaptive thresholds can be established based on simulation studies or specific system behaviors.
- **Examples**:
  - **CPU Utilization**: Safe threshold around 66%-70%.
  - **Network Traffic**: Thresholds are based on the maximum capacity of the network interface, applying the 66% rule.

Testing Availability

- **Failure Simulation**:
  - Generate random failures by stopping services or VMs to test system availability and recovery under stress.

Common Mistakes in Scalability Testing

- **Using CPU Stress Tools**:

    - Stress tools may not accurately reflect real-world loads when scaling, as they can terminate prematurely or cause uneven load distribution across VMs.
- **Incorrect Test Scenarios**:
    - For applications with heavy backend or database interactions, focusing only on front-end testing is insufficient.
    - A comprehensive mix of requests, including read/write operations, should be generated to evaluate system performance accurately.

Testing Repetition and Statistical Relevance

- **Test Repetition**:
    - Conducting a single test is insufficient for conclusive results. At least 20 tests should be performed, with averaged metrics providing the final assessment.

Cloud Storage

Introduction to Cloud Storage

The document begins by outlining the different forms of data storage in cloud systems. It covers:

- **Distributed File Systems:** Examples include Google File System (GFS) and Hadoop Distributed File System (HDFS), which are based on earlier concepts developed in Network File Systems and Parallel File Systems.
- **SQL Databases:** Traditional relational databases.
- **NoSQL Databases:** These include key-value storage systems like Google BigTable and Amazon Dynamo, designed for massive scaling, high availability, and simplified application development.

Basic Concepts and Atomicity

The document discusses the importance of atomicity in cloud storage systems:

- **Atomicity:** Ensures that a multi-step operation (transaction) completes without any interruption. This is essential for maintaining data integrity and consistency.
    - **Test-and-Set:** Writes to a memory location and returns the old content as non-interruptible operations.
    - **Compare-and-Swap:** Compares the contents of a memory location to a given value and modifies it if the two values match.

Types of Atomicity:

- **All-or-Nothing Atomicity:** Operations are divided into two phases—pre-commit (preparatory actions) and post-commit (irreversible actions). Logs are necessary to maintain the history of activities, ensuring consistency and managing failures.
- **Before-or-After Atomicity:** Ensures that the effect of multiple actions is as though they occurred one after another, maintaining external consistency.

Storage Models and Desired Properties

The document delves into physical storage models, which describe the layout of data structures on physical storage media:

- **Cell Storage Model:** Assumes storage cells of the same size, where each object fits exactly in one cell. It guarantees read/write coherence and before-or-after atomicity but does not guarantee all-or-nothing atomicity.
- **Journal Storage Model:** A more complex model that maintains a log of the entire history of a variable. It guarantees all-or-nothing atomicity by first recording actions in the log before modifying the cell store.

High-Performance, Scalable File Systems

Google File System (GFS)

- **Design:** GFS is designed to provide petabytes of storage using thousands of commodity components. It prioritizes **reliability** and handles large files, appends data rather than performing random writes, and operates with a relaxed consistency model.
  The most important features of the cloud access model are:
    - files range in size from a few GB to hundreds of TB

- *append* rather than *random write*
- s*equential* read
- *response time* is not the main requirement, data are processed in bulk
- relaxed consistency model to simplify the system implementation

- **File Structure:** GFS files are collections of fixed-size segments called chunks (64MB), which are stored on a Linux File System and replicated across multiple sites.
  Large chunks size:
  - to optimize performance for large files and to reduce the amount of metadata maintained by the system
  - to increase the likelihood that multiple operations will be directed to the same chunk
  - to reduces the number of requests to locate the chunk;
  - to maintain a persistent network connection with the server where the chunk is located
  - to reduce disk fragmentation
- **Cluster Architecture:** GFS uses a **master server** to control chunk servers, maintain metadata, and manage the system's reliability. The chunks are stored only in the control structure of the master's memory and are updated at system startup or when a new chunk server joins the cluster. This strategy allows the master to have up-to-date information about the location of the chunks.

  System reliability is a major concern. To minimize the recovery time, the master periodically checkpoints its state and at recovery time replays only the log records after the last checkpoint.
- **File Access and Write Protocol:** GFS handles file access through a primary chunk server responsible for updates. The write protocol involves multiple stages, including leasing, data transmission, and acknowledgments.

Hadoop Distributed File System (HDFS)

- **Architecture:** HDFS supports distributed applications and is optimized for handling large datasets. HDFS is a distributed fs written in Java and it replicates data on multiple nodes (3 by default) It uses a master/slave

architecture with NameNodes (masters) managing metadata and DataNodes (slaves) handling low-level read/write operations.

- **Write Protocol**
  - **Client Initiation**:
    - The client initiates the process by requesting the Namenode to create a new file. The Namenode checks for permissions and ensures that the file doesn't already exist.
  - **Block Allocation**:
    - The Namenode allocates blocks for the file and provides the client with a list of Datanodes where each block should be written. These Datanodes are chosen based on their availability and proximity.
  - **Pipeline Write**:
    - The client writes data to the first Datanode in the pipeline. This Datanode then forwards the data to the next Datanode, and so on, until all replicas of the block are written. This process ensures that data is replicated across multiple nodes as it is being written.
  - **Ack Mechanism**:
    - Each Datanode in the pipeline acknowledges the receipt of the block to the previous node. Once the last Datanode in the chain has received the block, it sends an acknowledgment back to the client through the pipeline, ensuring that the write process is complete.
  - **Error Handling**:
    - If a Datanode fails during the write process, the pipeline is restructured, and the data is written to a new Datanode. The Namenode is updated with this new block information.
  - **File Closure**:
    - After all data blocks are written and acknowledged, the client closes the file. The Namenode then finalizes the file creation in the file system’s metadata.

This protocol ensures data reliability and fault tolerance during the write process by replicating data across multiple nodes and using an acknowledgment mechanism to confirm successful writes.

- **Read Protocol**
  - **Client Interaction**:
    - When a client wants to read a file, it communicates with the Namenode to obtain the metadata, which includes the locations of the data blocks that make up the file.
  - **Data Block Locations**:
    - The Namenode provides the client with a list of Datanodes that store replicas of each block. The client chooses the nearest or most appropriate Datanode to read from, usually based on network topology and load balancing considerations.
  - **Pipeline Read**:
    - The client reads the data blocks directly from the chosen Datanodes. The process is efficient, leveraging the high bandwidth of the cluster, and involves reading from multiple Datanodes in parallel if needed.
  - **Checksum Validation**:
    - As the client reads data, it performs checksum validation to ensure data integrity. If a block's checksum does not match, the client attempts to read the block from another replica.
  - **Sequential and Streaming Reads**:
    - HDFS is optimized for large, sequential read operations, meaning that it efficiently streams large files to clients.

NoSQL Databases

OLTP - Transaction processing

Many cloud applications are based on *online transaction processing* (**OLTP**) and operate under tight latency constraints. Relational databases are not able to handle the massive amount of data and especially the real-time demand.

Major concerns in OLTP system are:

- Response time → memcaching is a solution
- Scalability → vertical scaling always possible, horizontal scaling may cause issues of consistency among multiple distributed copies of the DB

Document stores and *NoSQL* databases are designed to scale well, do not exhibit a single point of failure and have a built-in support for consesus-based decisioning.

The approach in the design of the NoSQL databases allows data to be inconsistent and implementation of the ACID properties is on the application developer shoulders.

BigTable (Google)

Bigtable is a distributed storage system developed by Google, designed to be scalable, high-performance, and highly available. It supports a wide variety of workloads, from batch processing to latency-sensitive tasks.

- **Data Model**: Bigtable uses a sparse, distributed, persistent multidimensional sorted map, indexed by a row key, column key, and timestamp. Data is treated as uninterpreted strings, giving clients control over data layout and format.
  - **Rows**: Rows are indexed by a key and are lexicographically ordered. This ordering allows efficient access and dynamic partitioning into tablets (the units for load balancing). Read of short ranges are efficient and typically require communication with only a small number of machines.
  - **Columns**: Column key are grouped into sets called **column families**, with each column family containing data of the same type. Column keys follow the format `family:qualifier`. The system is designed for efficient compression and access control at the family level.
  - **Timestamps**: Each cell in Bigtable can hold multiple versions, indexed by timestamps, allowing for historical data retrieval. Different version of a cell are stored in decreasing timestamp order (the most recent can be read first). Automatic garbage collection of old versions is supported.
- **Building Blocks**:
  - **Google File System (GFS)**: Bigtable relies on GFS for storing its data.
  - **Google SSTable**: A file format used internally by Bigtable, providing a persistent, ordered immutable map from keys to values. Efficient access is ensured via block indices loaded into memory. Each SSTable contains a **sequence of blocks** and a **block index** to locate blocks.

- **Chubby**: A distributed lock service used for synchronizing Bigtable operations, such as ensuring that there is at most one active master at a time, storing the bootstrap location of Bigtable data, discovering tablet servers and finalize tablet server deaths, storing Bigtable schema informations and finally storing access control lists.

- **Architecture**:

  Consist of one **master server** and many **tablet servers.** A BigTable cluster store a number of tables, each table consists in a set of tablets.

  The master is responsible to assigning tablets to tablet servers, detecting the addition and expiration of tablet servers, balancing tablet-server load and garbage collectors, handling schema changes such as table and column family creations.

  Tablet servers can be dynamically added/removed from a cluster to accomodate the workload. Each tablet server manages a set of tablets, handles read and write requests to the tablets that it has loaded and splits tablets that have grown too large.

Dynamo (Amazon)

- **Design:** Dynamo is a highly available and scalable key-value store designed for Amazon's platform. It is an eventually consistent data store, that is all updates reach all replicas eventually. Optimistic replication techniques are used to increase availability in an environment prone of failures.

  In optimistic replications, changes are allowed to propagate to replicas in the background. A read/write is successful if a min number of node can complete successfully the operation.

  Amazon’s Dynamo need some requirements:

  - Query model
  - ACID properties
  - Efficiency
  - No specific security requirements
  - Scalability to hundred of storage hosts

- **Key Design Principles:**

- **Data Replication:** Data is replicated across multiple hosts to ensure availability and durability. The system uses optimistic replication techniques, resolving conflicts either at write time or read time.
- **Incremental scalability:** Be able to scale out one storage host at a time.
- **Symmetry:** Every node in Dynamo should have the same set of responsiblites as its peers.
- **Decentralization:** Favore decentralized peer-to-peer techniques over centralized control.
- **Heterogeneity:** To exploit heterogeneity in the infrastructure.

- **System Architecture:**
  - **System Interface:**
    - **Get(key)** locates the object replicas associated with the key in the storage system and returns a single object or a list of objects with conflicting versions along with a context.7
    - **Put(key, context, object)** determines where the replicas of the object should be placed based on the associated key, write the replicas to disk
  - **Partitioning Algorithm:**
    - Dynamo uses consistent hashing and virtual nodes to distribute load evenly across storage hosts. Each node in the system is assigned a random value within this space which represents its "position" on the ring. Each data item identified by a key is assigned to a node by hashing the data item's key to yield its position on the ring
  - **Data Versioning:**
    - Updates are treated as new versions, marked by vector clocks to capture causality. This allows the system to handle divergent versions of data and reconcile them as needed.

Object Storage: Amazon S3

- **Storage Model:**

    - Data stored as named “objects”that are grouped in named “buckets”. Each object is identified by a unique key, and operations are performed using the PUT and GET APIs.
- **Data Consistency:** S3 provides strong read-after-write consistency for both PUT and DELETE requests. However, updates to bucket configurations follow an eventual consistency model, and S3 does not support object locking for concurrent writers.

Failure Handling in S3

S3 manages read and write failures using a retry mechanism. Clients are responsible for retrying failed requests until they succeed. S3 also uses MD5 checksums (returned in the ETag field) to verify the integrity of data after it is written or read.

Examples of Concurrent Application Behavior

- **Concurrent Writes:** S3 uses a last-writer-wins approach to resolve conflicts between simultaneous PUT requests to the same key. The order of operations is not predictable, so applications must perform a read after both writes to determine the final value.

Oral Exam Questions

Virtualization

- **Hypervisor**
    - Communication/resource allocation
    - Essential components/functions
    - Filtering of instructions
- **Supervisor**
    - Kernel mode
    - User mode
    - Containers management

- **Wrapper for calls**
  - Allocator

Writes in HDFS

- Write operations in the file system

Orchestration vs Choreography

File Partitioning (key-value)

AWS Threshold on CPU Utilization

- Cloudwatch: average aggregate value
- Autoscaling in AWS
  - **Target scaling** → Choose a target (Avg CPU utilization ecc) and automatically scale up/down
  - **Simple/step scaling** → Choose a threshold (or many thresholds), set the number of instances to be settled. NO COOLDOWN (only for step scaling)
  - **Scheduled scaling** → Set a time boundary for scaling (E.g. every day at 8pm)

SOA vs Microservices

Scalability

- Ways to scale: vertical, horizontal

BigTable

- Automation
- Container

Cloud Brokers

- Their role

Characteristics of Cloud

- Broad access network

- Rapid elasticity
- Resource pooling
- Measured service
- Automation, security
- Availability, easy maintenance

Deployment Models

- Private, Public, Community and Hybrid

DynamoDB

Containers

- Cgroups, chroot
- Technologies: building blocks, namespace

Autonomic Systems

- MAPE-K
- Self properties

Autoscaling Algorithms

- Threshold-based
- Model-based
- Proactive

Kubernetes

- Scheduling of Pods
- Description

All types of memory provided by Docker

- **Tmpfs** è utile per memorizzare dati temporanei in memoria, senza persistenza.
- **Volumes** offrono una soluzione robusta e persistente per la gestione dei dati tra container, indipendenti dal filesystem dell'host.

- **Direct Data Mount** (bind mounts) consente di accedere direttamente ai file dell'host, ideale per situazioni specifiche dove il container deve interagire con file dell'host.
- **Writable Layer** viene usato implicitamente in ogni container, ma i dati vengono persi quando il container viene eliminato.

Big Data & Cloud Technologies: Questions

1. Big table storage: structure and how data is indexed
2. Automation properties
3. Container technologies
4. How does a client write in Hadoop filesystem?
5. Services provided by the cloud broker
6. Three ways of scaling in microservices
7. Do containers share resources in pods?

HDFS & Virtualization: Questions

1. Which protocol does HDFS use and how does it work? (architecture)
2. HDFS vs GFS
3. Virtualization: the main three components
4. How does the hypervisor communicate/allocate additional resources?

Miscellaneous Questions

- The three main components of the supervisor (???)
- Describe Dynamo's partitioning algorithm and how replication works
- How does GFS work? (better understanding than what's written in Simone's notes)
- What is a cloud broker? (only part I didn’t remember from notes)
- Bindmount vs volume in Docker: which is more efficient in terms of portability? And in terms of speed?